Coding Helper

Scanner

```
matches = []
for asset_id, services in services_by_asset.items():
    for service in services:
        for sig in signatures:
            if service["name"] != sig["service_match"]:
                continue
            if version_is_below(service["version"], sig["vulnerable_below"]):
                matches.append((asset_id, service, sig))
return matches
```

```
  parts = []
  for segment in version_str.split("."):
      digits = ""
      for ch in segment:
          if ch.isdigit():
              digits += ch
          else:
              break
      parts.append(int(digits) if digits else 0)
  return tuple(parts)
```

```
return {
    "plugin_id": signature["plugin_id"],
    "plugin_name": signature["plugin_name"],
    "severity": signature["severity"],
    "host": asset["hostname"],
    "asset_id": asset["asset_id"],
    "ip": asset["ip"],
    "port": service["port"],
    "protocol": service["protocol"],
    "service": service["name"],
    "installed_version": service["version"],
    "cve": signature["cve"],
    "cvss_vector": signature["cvss_vector"],
    "cvss_base_score": signature["cvss_base_score"],
    "description": signature["description"],
    "solution": signature["solution"],
}
```

Calculator

```
cvss_scaled = normalize_cvss_to_scale(finding["cvss_base_score"])
epss_scaled = normalize_epss_to_scale(epss_score)
return (
    cvss_scaled * LIKELIHOOD_WEIGHTS["cvss"]
    + epss_scaled * LIKELIHOOD_WEIGHTS["epss"]
    + exploit_availability * LIKELIHOOD_WEIGHTS["exploit_availability"]
)
```

```
return (
    asset["asset_criticality"] * IMPACT_WEIGHTS["asset_criticality"]
    + asset["data_sensitivity"] * IMPACT_WEIGHTS["data_sensitivity"]
    + asset["business_impact"] * IMPACT_WEIGHTS["business_impact"]
)
```

```
if epss_score >= 0.75:
    return 5
if epss_score >= 0.5:
    return 4
if epss_score >= 0.25:
    return 3
if epss_score >= 0.1:
    return 2
return 1
```

```
if cvss_base_score >= 8:
    return 5
if cvss_base_score >= 6:
    return 4
if cvss_base_score >= 4:
    return 3
if cvss_base_score >= 2:
    return 2
return 1
```

Heatmap

```
bucket = round(score)
return max(1, min(5, bucket))
```

```
grid = [[[] for _ in range(5)] for _ in range(5)]
for finding in register:
    li = bucket_score(finding["likelihood_score"])
    im = bucket_score(finding["impact_score"])
    label = f"{finding['host']}: {finding['plugin_name']}"
    grid[im - 1][li - 1].append(label)
return grid
```